OBJECT-ORIENTED PROGRAMMING

CL-1004

LAB 04

Dive into Constructors, "`this`" Keyword, Type of Constructors, Member Initialization List, and Destructors

National University of Computer and Emerging Sciences–NUCES – Karachi

Instructor: Talha Shahid

Contents

1. Constructors

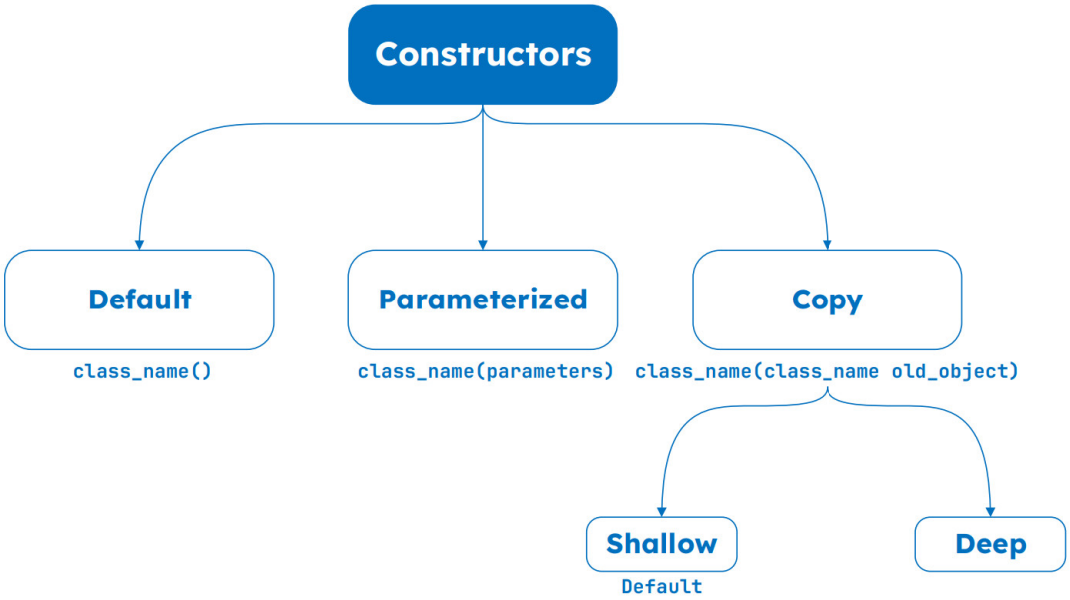
Constructor is the special type of member function in C++ classes. It is automatically invoked when an object is being created. It is special because its name is same as the `class` name. A constructor can be used for the following functions:

1. To initialize data member of class: In the constructor member function (which the programmer will declare), we can initialize the default vales to the data members and they can be used further for processing.
2. To allocate memory for data member: Constructor is also used to declare run time memory (dynamic memory for the data members).

Constructor has the following properties:

- Constructor has the same name as the `class` name.
- The name is case sensitive.
- Constructor does not have any `return` type.
- We can overload constructor; it means we can create more than one constructor of class.
- It must be `public` type (declared inside the public access modifier in the class).

2. Types of Constructors



Let us understand the types of constructors in C++ by taking a real-world example.

Suppose you went to a shop to buy a marker. When you want to buy a marker, what are the options.

- You go to a shop and say give me a marker.

So just saying give me a marker mean that you did not set

- which brand name
- and which color,

you didn't mention anything, you said that you want a marker.

So, when we said just, I want a marker so whatever the frequently sold marker is there in the market or in his shop he will simply hand over that. And this is what a **Default Constructor** is!

The second method is you go to a shop and say:

- I want a marker, red in color
- and XYZ brand.

So, you are mentioning this and he will give you that marker. So, in this case you have given the parameters and this is what a **Parameterized Constructor** is! Then the third one you go to a shop and say I want a marker like this (a physical marker on your hand). So, the shopkeeper will see that marker. Okay, and he will give a new marker for you. So, copy of that marker. And that's what a **Copy Constructor** is!

2.1. Default Constructor

Default constructor is the constructor, which does not take any argument. It has no parameters.

```cpp
#include <iostream>
using namespace std;

class Product {
private:
    string name;
    double price;

public:
    // default Constructor
    Product() {
        name = "Unknown";
        price = 0.0;
    }

    void addProduct(const string& productName, double productPrice){
        name = productName;
        price = productPrice;
        cout << "Product added: " << name << " - $" << price << endl;
    }

    // method to update price
    void updatePrice(double newPrice){
```

```
        price = newPrice;
        cout << "Updated price of " << name << " to $" << price << endl;
    }

    // method to display product details
    void display(){
        cout << "Product: " << name << ", Price: $" << price << endl;
    }
};

int main() {

    Product product; // creating a Product object using default constructor

    product.display(); // unkown product

    product.addProduct("Laptop", 1200.0); // adding a new product
    product.updatePrice(1100.0); //updating price

    product.display(); // new product

    return 0;
}
```

Output	Product: Unknown, Price: $0 Product added: Laptop - $1200 Updated price of Laptop to $1100 Product: Laptop, Price: $1100

2.2. Parametrized Constructor

It is possible to pass arguments to constructors. Typically, these arguments help initialize an object when it is created. To create a parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor's body, use the parameters to initialize the object.

```
#include <iostream>
using namespace std;

class Product {
private:
    string name;
    double price;

public:
    // default Constructor
    Product(){
        name = "Unknown";
        price = 0.0;
    }
```

```
    // parameterized Constructor
    Product(string productName, double productPrice){
        name = productName;
        price = productPrice;
        cout << endl << "Product added: " << name << " - $" << price << endl;
    }

    // method to update price
    void updatePrice(double newPrice){
        price = newPrice;
        cout << "Updated price of " << name << " to $" << price << endl;
    }

    // method to display product details
    void display(){
        cout << endl << "Product: " << name << ", Price: $" << price << endl;
    }
};

int main() {

    Product defaultProduct; // creating a Product object using default
constructor

    defaultProduct.display();

    Product laptop("Laptop", 1200.0); // creating a Product object using
parameterized constructor

    laptop.updatePrice(1100.0);
    laptop.display();

    return 0;
}
```

Output	Product: Unknown, Price: $0 Product added: Laptop - $1200 Updated price of Laptop to $1100 Product: Laptop, Price: $1100

2.3. Copy Constructor

A copy constructor is a member function, which initializes an object using another object of the same class. The copy constructor in C++ is used to copy data of one object to another.

2.3.1. Shallow Copy Constructor

A shallow copy copies the memory addresses of the original object's data members to the new object without allocating new memory. This is the default behavior when a copy constructor is not explicitly defined.

```cpp
#include <iostream>
using namespace std;

class Shallow {
public:
    int* data;

    // constructor
    Shallow(int value){
        data = new int(value);
    }

    // shallow Copy Constructor
    Shallow(Shallow& obj){
        data = obj.data; // copies the pointer, NOT the actual value
    }

    void display(){
        cout << "Data: " << *data << ", Address: " << data << endl;
    }
};

int main() {
    Shallow obj1(10);
    Shallow obj2(obj1); // shallow copy

    obj1.display();
    obj2.display();

}
```

Output	Data: 10, Address: 0x1016da0 Data: 10, Address: 0x1016da0

2.3.2. Deep Copy Constructor

A deep copy allocates new memory and copies the actual data from the original object instead of just copying the memory address. This prevents shared memory issues.

```cpp
#include <iostream>
using namespace std;

class Deep {
public:
    int* data;

    // constructor
    Deep(int value) {
        data = new int(value);
    }

    // deep Copy Constructor
    Deep(Deep& obj){
        data = new int(*obj.data); // allocates new memory and copies value
```

```
    }

    void display(){
        cout << "Data: " << *data << ", Address: " << data << endl;
    }

};

int main(){
    Deep obj1(20);
    Deep obj2(obj1); // deep copy

    obj1.display();
    obj2.display();

}
```

Sample Run	Data: 20, Address: 0x10d6da0 Data: 20, Address: 0x10d6db0

3. Destructor

A destructor is also a special member function as a constructor. Destructor destroys the class objects created by the constructor. Destructor has the same name as their class name preceded by a tilde (~) symbol. It is not possible to define more than one destructor.

The destructor is only one way to destroy the object created by the constructor. Hence destructor can-not be overloaded. Destructor neither requires any argument nor returns any value. It is automatically called when the object goes out of scope.

Destructors release memory space occupied by the objects created by the constructor. In destructor, objects are destroyed in the reverse of object creation.

```
#include <iostream>
using namespace std;

class MyClass{
public:
    MyClass() {
        cout << "Constructor called! Object created." << endl;
    }

    // destructor
    ~MyClass(){
        cout << "Destructor called! Object destroyed." << endl;
    }

    void display(){
        cout << "Hello from MyClass!" << endl;
    }
};

int main() {
```

```
    MyClass obj;
    obj.display();

    // obj goes out of scope here, and the destructor is called automatically

    cout << "Object is now out of scope." << endl;

}
```

Sample Run	Constructor called! Object created. Hello from MyClass! Object is now out of scope. Destructor called! Object destroyed.

4. this Keyword

this keyword is a pointer to the current object. It is used to refer to the member variables and member functions of the current object. It can be particularly useful when there is a naming conflict between a member variable and a local variable or parameter with the same name in a member function. By using this, you can disambiguate between the two and access the member variable specifically.

```
#include <iostream>
using namespace std;

class Product {
private:
    string name;
    double price;

public:
    // default Constructor
    Product(){
        name = "Unknown";
        price = 0.0;
    }

    // parameterized Constructor
    Product(string name, double price){
        this->name = name;
        this->price = price;
        cout << endl << "Product added: " << name << " - $" << price << endl;
    }

    // method to update price
    void updatePrice(double newPrice){
        price = newPrice;
        cout << "Updated price of " << name << " to $" << price << endl;
    }

    // method to display product details
```

```
    void display(){
        cout << endl << "Product: " << name << ", Price: $" << price << endl;
    }
};

int main() {

    Product defaultProduct; // creating a Product object using default
constructor

    defaultProduct.display();

    Product laptop("Laptop", 1200.0); // creating a Product object using
parameterized constructor

    laptop.updatePrice(1100.0);
    laptop.display();

    return 0;
}
```

Output	Product: Unknown, Price: $0 Product added: Laptop - $1200 Updated price of Laptop to $1100 Product: Laptop, Price: $1100

5. Member Initializer List

In C++, a member initializer list is used to initialize class members directly when an object is created. It is written after the constructor's parameter list and before the constructor's body. This approach is especially useful for constant variables, reference variables, and base class initialization. Instead of assigning values inside the constructor body, we can directly initialize the variables in the initializer list. It is written after the constructor's parameter list and before the constructor body.

This method is very useful for initializing constant variables, reference variables, and base class members because they must be set at the time of object creation. It also helps in making the code faster and more efficient by avoiding extra assignments inside the constructor.

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    int length;
    int width;

public:
    // constructor using member initializer list
    Rectangle(int l, int w) : length(l), width(w) {
```

```
        cout << "Rectangle object created with length = " << length
             << " and width = " << width << endl;
    }

    int area() {
        return length * width;
    }

    // destructor
    ~Rectangle() {
        cout << "Rectangle object destroyed." << endl;
    }
};

int main() {

    Rectangle rect(5, 10);
    cout << "Area of the rectangle: " << rect.area() << endl;

    return 0;
}
```

Sample Run	Rectangle object created with length = 5 and width = 10 Area of the rectangle: 50 Rectangle object destroyed.